

In a distributed banking environment, customers access their accounts and conduct transactions (deposits, withdrawals, and queries) across multiple branches of the bank. Each branch operates as an independent replica with a local balance, which must remain consistent with the balances at other branches.

The goal of this project is to model a distributed banking system using Python and gRPC to simulate interactions between customers and branch processes and the interactions between branch processes to maintain replication consistency. Each customer should be able to perform transactions seamlessly, and branch replicas must synchronize to reflect accurate balances across all locations.

To set up the environment to implement this project, we had to use python's `grpcio` (version 1.67.0) and `grpcio-tools` (version 1.67.0) packages. The `protobuf` version accompanying the `grpcio-tools` was version 5.28.3. I additionally downloaded a `protobuf 3` extension to my `vscode` setup for syntax validation. I then started implementing the project by defining the services needed to support the interactions between customer and branch, and branch and branch.

To support the messages, I defined an interface enum (it defined the types of transactions), and a result enum. The interface enum had fields `WITHDRAW`, `DEPOSIT`, and `QUERY`. The result enum had fields `SUCCESS` and `FAILURE`. I then defined messages `CustomerMessage`, `BranchMessage`, and `Reply`. `CustomerMessage` are messages intended only to be sent by customers to a branch. It consisted of the fields `id` (for customer id), `interface` (type of transaction a customer wished to perform), and `amount` (even though all fields is optional by default in a proto message, I wanted to clearly outline that amount is optional as it is only used in tandem with `WITHDRAW` and `DEPOSIT` interfaces). `BranchMessage` are messages intended only to be sent between branches. It consisted of the fields `interface` (type of transaction to be performed at a branch) and `amount`. `amount` is not optional in this message because I designed the gRPC service such that banks only communicate with each other to propagate withdraw or deposit transactions. The `Reply` message is a message used to reply to both customers and branches. It consists of the fields `interface`, `result` (indicates whether a request was successful or not), and `balance` (optional, is only returned when the interface is query).

The two services I defined were `Query` and `Propagate`. `Query` is intended for customer to branch communication (customer sends a query request with `CustomerMessage` as argument and expects a `Reply` message). `Propagate` is intended for branch to branch communication (branch sends a propagate request with `BranchMessage` as argument and expects a `Reply` message).

To be able to support customer requests, the `Branch` class defined in `branch.py` has `_query`, `_withdraw`, and `_deposit` methods as well as `Query`. When a branch object receives a `Query` rpc, it first checks the interface of the `CustomerMessage` received. If it is `QUERY`, it creates a `Reply` message consisting of the interface initially received, the result of the request,

and the balance of the branch. If the interface is either WITHDRAW or DEPOSIT, the branch propagates these requests to the other branches by first establishing communication channels to the list of branches the current branch is aware of (including itself) and sending a Propagate request to all those branches. When a branch receives a Propagate RPC, it credits or debits (depending on the interface) its balance as per the amount in the BranchMessage. It then responds with a Reply message. If at least one of the responses is a failure, `_withdraw` and `_deposit` would return false and we would construct a Reply response with its result set to Failure. (nb. Since this is a simplified model, proper transaction semantics were not implemented. In the case of a proper model, all requests completed prior to the failure would have to be reversed.) If all communications with other branches in a branch's list of branches are successful, `_withdraw` and `_deposit` would return true and we would construct a Reply response consisting of the request interface and the result of the RPC call (successful). If the interface of a request isn't any of the three discussed, we respond with a failed Reply message.

The customer class defined in `customer.py` consists of the methods `createStub`, `executeEvents`, and `returnMsgs`. `createStub` establishes a communication channel to a branch on a server in another process. All branches are on processes connecting to port `50,000+id` (this is to ensure that the ports used are very unlikely to be used by other processes when the servers are initially set up). Thus if a customer wants to communicate with the branch with its corresponding id, it can do so on port `50,000+self.id`. `executeEvents` first calls `createStub`. It then iterates over the list of events a customer ought to perform (this is stored in `events` as a list of dictionaries). For each event, we perform an RPC to a corresponding branch depending on the interface of the event. After this, I then construct the outcome of the event depending on the interface and then store it in the `recvMsg` class field before closing the communication channel.

To run the servers, we first read in the json file containing the input. From there, we retrieve each branch's set up information and a list of all branch ids. When we create a branch server, we first create a Branch object with the branch id, the starting balance, and the ids of the branches it can communicate with. We then instantiate a gRPC server and add our branch service as a servicer. We then connect our server to a port from which it can listen to requests from before starting the server. I used python's multiprocessing module to instantiate each server on a different process and have it listen on ports according to their id (all branches are on processes connecting to port `50,000+id`).

To run the clients, we first read in the json file containing the input. From there, we retrieve each client's information and return a list of Customer objects using the information we retrieved. I ran the clients in sequential order on a single process. For each customer object, I called the `executeEvents` method and stored the results to be written to the output json file.

The results stored in the output json file consist of the id of the communicating customer and the responses received for each action then performed (namely a deposit/withdraw followed

by a query). The first 50 actions performed by customers with id 1 to 50 are deposit actions with an amount of 10. Since the initial balance was 400, the last deposit action leaves the bank balance across all branches to be 900. The ensuing 50 actions performed by the same customers are withdrawals of an amount of 10. The last withdrawal action leaves the bank balance across all branches to be 400, i.e. back to the initial balance. In both sets of 50 actions, the customer ids are in increasing order.